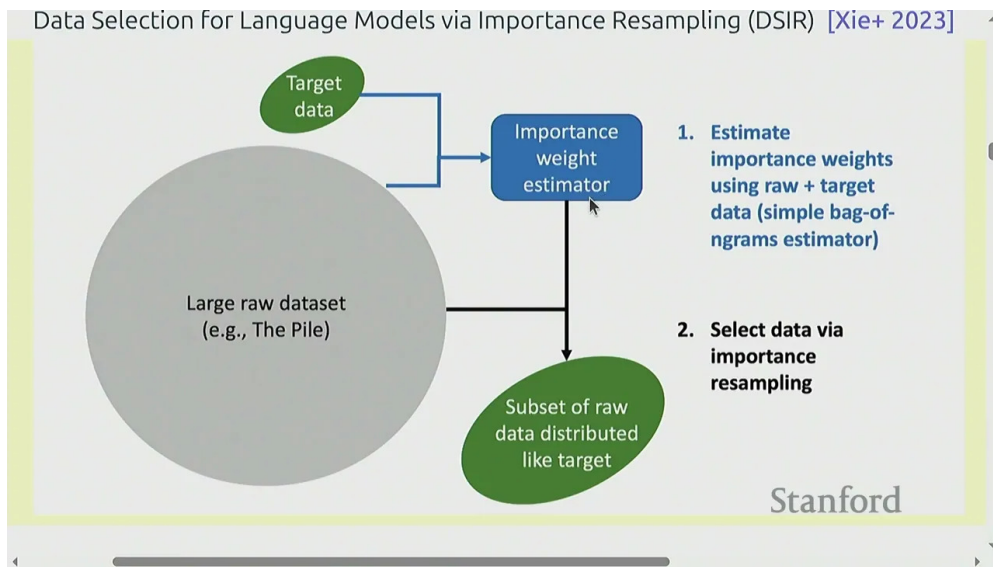


课程笔记

CS336 Lecture 14: 数据过滤与去重

基于授课内容整理

2025 年春季



视频作者/频道: Stanford CS336

发布日期: 2025 年春季

视频时长: 约 30 分钟

视频链接: 未填写

目录

1 导言：数据不是从天上掉下来的	2
2 过滤算法：从目标数据到可打分的原始数据	2
2.1 问题抽象	2
2.2 三种代表性实现	2
2.3 KenLM 与 n-gram 语言模型	2
2.4 fastText 分类器	3
2.5 DSIR: Importance Resampling	3
3 同一套机械的三种用途	4
3.1 语言识别	4
3.2 质量过滤	4
3.3 毒性过滤	5
4 去重：从精确重复到近重复	5
4.1 两类重复	5
4.2 哈希函数	5
4.3 精确去重	5
4.4 Bloom Filter	6
4.5 Jaccard 相似度	6
4.6 MinHash	6
4.7 Locality Sensitive Hashing	7
5 把这一切统一起来	7
6 本章小结	8

1 导言：数据不是从天上掉下来的

上一讲回顾了训练语言模型时使用的数据集，从早期语料到现代的大规模网络语料，核心结论是：**高质量数据往往需要主动构造**。真实世界的数据通常先存在于活跃服务里，之后要经历抓取、导出、HTML 转文本、语言过滤、质量过滤、毒性过滤、去重等一系列处理，最后才会成为训练语料。

本讲的主线

1. 给定少量目标数据 T 和大量原始数据 R ，如何挑出与 T 相似的子集。
2. 如何把同一套过滤机制应用到语言识别、质量过滤和毒性过滤。
3. 如何在线性时间内做大规模去重，尤其是近重复检测。

从工程角度看，这一讲的关键词不是“最强模型”，而是“足够快、足够便宜、足够稳定”。在数据筛选里，**粗糙但高吞吐**的办法通常比精细但昂贵的方法更有价值。

2 过滤算法：从目标数据到可打分的原始数据

2.1 问题抽象

可以把过滤写成一个非常抽象的任务：给定目标数据 T 和大量原始数据 R ，希望找到一个子集 $T' \subset R$ ，使得 T' 和 T “相似”，但又不是简单复制 T 。

过滤算法应该满足什么

- **能泛化**：不能只是记住 T ，而是要学到“像 T 的东西”。
- **足够快**：要能跑在整个网页级别的数据上。
- **可打分**：最好把每个样本映射成一个分数，然后按阈值保留或重采样。

这类任务的共同模式非常稳定：先从目标数据估计一个模型或打分函数，再在原始数据上计算分数，最后根据分数做筛选。

2.2 三种代表性实现

方法	打分方式	优点	局限
KenLM / n-gram	$p_T(x)$ 或 perplexity	极快，简单	粗糙，局部上下文
fastText	$p(T x)$	训练和推理都快	仍然偏判别式
DSIR	$p_T(x)/p_R(x)$	更接近分布匹配	仍依赖简化建模

表 1: 三类过滤方法都在做“先建模，再打分，再筛选”

2.3 KenLM 与 n-gram 语言模型

n-gram 语言模型是最经典的过滤基线。对于三元模型，最大似然估计写成

$$p(w_i | w_{i-2}, w_{i-1}) = \frac{\text{count}(w_{i-2}, w_{i-1}, w_i)}{\text{count}(w_{i-2}, w_{i-1})}.$$

它的优点很直接：**就是计数再归一化**。真正麻烦的是稀疏性。随着 n 变大，许多合理的 n -gram 根本从未在语料中出现，于是就需要平滑。课程里提到的 Kneser-Ney smoothing 就是处理“没见过的 n -gram”最常见的办法之一。

Kneser-Ney 平滑的直觉

如果高阶上下文计数不够，就回退到低阶上下文；如果低阶也不可靠，就继续 back off。它的核心不是“凭空发明概率”，而是把统计信心不足的部分交给更短的上下文。

KenLM 是一种常用的高性能实现。它原本常见于机器翻译，但在数据过滤中也很流行，因为它足够快，能把大规模文本打成一个粗糙但有用的分数。课堂里的 CCNet 例子就是把段落按 perplexity 排序，保留前 1/3，用来构造后来的 LLaMA 数据集。

n -gram 模型的边界

它能很好地抓住“像不像自然文本”，但很难理解长程语义、篇章结构或事实正确性。换句话说，它更适合过滤“明显不像人话”的内容，而不是判断一篇文章是否真的高质量。

2.4 fastText 分类器

fastText 的核心思路是把文本表示成词袋或 n -gram 词袋，再做线性分类。它之所以常见，是因为它把一个高维稀疏问题压缩成了一个非常轻量的模型。

如果直接做词袋分类，参数规模可能接近 $V \times K$ 。fastText 通过“词嵌入平均 + 线性头”的方式把参数量降到 $H(V + K)$ 左右，其中 $H \ll V$ 。

fastText 为什么适合过滤

- 推理快，适合扫描海量网页。
- 训练也快，适合快速迭代规则和阈值。
- 对二分类过滤尤其自然：好/坏、目标/非目标、英语/非英语。

它还支持 bag of n -grams，并使用 hashing trick 避免 n -gram 维度爆炸。代价是哈希冲突，但在粗粒度过滤里这通常是可以接受的。

2.5 DSIR: Importance Resampling

DSIR 的思路更接近“分布匹配”。设目标分布为 p ，原始分布为 q ，则重要性权重是

$$w(x) \propto \frac{p(x)}{q(x)}.$$

先从 q 中采样，再按权重重采样，最后得到更接近 p 的样本集合。

这里的障碍在于：目标数据通常太少，没法直接拟合一个复杂分布。所以 DSIR 采用 hash n -gram 的方式，把文本映射到低维离散特征空间，然后估计一个简化版的分布。

DSIR 的意义

和“只做分类”相比，显式建模 p/q 更接近“我要什么数据”的原始目标，因此在多样性和覆盖性上通常更自然。

3 同一套机械的三种用途

3.1 语言识别

语言识别的目标很直接：给定文本，判断它是不是目标语言，例如英语。这里的关键不是“是否能分多语言”，而是“是否能高效地筛掉非目标语言”。

为什么不直接做一个巨型多语种模型？因为在 compute 限制下，若你把很多算力花在不关心的语言上，目标语言的质量可能会被稀释。BLOOM 只有约 30% 英语训练比例就是一个典型例子。

语言识别的常见困难

- 短文本信息不足。
- 低资源语言训练数据少。
- 方言和变体容易被误杀。
- 相近语言之间容易混淆。
- code-switching 的“真值”本身就不稳定。

课程里提到的 fastText 语言识别模型支持 176 种语言。Dolma 的做法很朴素：保留英语概率足够高的网页。OpenMathText 则更进一步，把规则过滤、KenLM perplexity、fastText 数学写作分类器组合起来，最终构造出一个高质量的数学语料库。

3.2 质量过滤

质量过滤是一个更模糊的概念。它并不只是“语言通顺”这么简单，但在大规模网络语料里，它往往首先意味着：去掉垃圾页、模板页、明显重复内容、低信息密度内容。

代表工作	正样本	负样本
GPT-3	Wikipedia / WebText2 / Books	CommonCrawl
LLaMA / RedPajama	Wikipedia 引用页面	CommonCrawl
phi-1	教材 + GPT 生成的高价值代码样本	低价值代码样本

表 2: 几种常见的质量过滤思路

GPT-3 的做法是训练线性分类器，再根据分数进行随机保留。LLaMA 和 RedPajama 更强调“被维基百科引用的页面更可靠”。phi-1 则把“教育价值”当成过滤目标，用少量高质量样本去筛大规模代码数据。

质量过滤的本质

它不是在定义“完美文档”，而是在建模“哪些数据更值得被训练”。因此，哪怕模型很粗，只要能持续过滤掉最差的那一部分，收益就可能很大。

3.3 毒性过滤

毒性过滤通常在数据集清洗里承担更直接的治理角色。课程里提到的 Dolma 使用了 Jigsaw Toxic Comments 数据集训练分类器，把评论分成诸如 toxic、severe_toxic、obscene、threat、insult、identity_hate 等类别。

在工程上，这类模型常常是 fastText 级别的轻量分类器，因为它要跑在大规模网页上。课程里还提到对 hate 和 NSFW 分别训练分类器，然后用它们做过滤或标记。

毒性过滤的注意事项

它和质量过滤一样，都会遇到误杀问题。对于短文本、俚语、引号引用、讽刺、代码混排等情形，分类器很容易把“上下文不完整”误认为“有毒”。

4 去重：从精确重复到近重复

4.1 两类重复

去重通常分成两种：

- **精确重复**：完全一样，常见于镜像站、GitHub fork、重复抓取。
- **近重复**：只差少量 token，例如模板化文本、条款、轻微排版差异。

这两个问题看起来简单，但规模一大就会变成“比较所有和所有”的二次方灾难。所以去重的核心诉求是：**线性时间或近线性时间**。

4.2 哈希函数

哈希函数把大的 item 映射到更小的值。它的好处是快、占用空间小，但也会碰撞。对去重而言，这通常足够了，因为我们不是在做密码学，而是在做大规模工程筛选。

常见哈希选择

- SHA-256：抗碰撞强，但慢。
- DJB2 / MurmurHash / CityHash：快，但不追求密码学抗碰撞。

课程里统一使用 MurmurHash。原因也很朴素：够快、够常见、足够适合哈希表和近似集合运算。

4.3 精确去重

精确去重可以直接按 hash 分组，然后每组保留一个样本。用 MapReduce 风格写，会非常容易并行化。

对于 C4 这样的语料，常见做法是按 3-sentence span 做精确去重。这样做的好处是简单直接，但代价是可能把文档中间的一段删掉，导致剩余文档不连贯。

精确去重的优缺点

- 优点：简单、清晰、高精度。
- 缺点：无法处理只差几个 token 的近重复。

4.4 Bloom Filter

Bloom filter 是一个近似集合成员查询结构，适合“有没有见过这个 item”这类问题。它的特性是：

- 很省内存；
- 可以插入，但不方便删除；
- 返回“没有”时一定没有；
- 返回“有”时通常是有，但可能是假阳性。

如果用 m 个桶、 k 个哈希函数、插入 n 个元素，假阳性率常写成近似：

$$f \approx (1 - e^{-kn/m})^k.$$

在固定 m/n 比例下，最优的哈希函数数目大约是

$$k \approx \frac{m}{n} \ln 2.$$

Bloom filter 的本质

它用“少量误报”换“极大的空间节省”。对于网页级去重，这通常是合理交易，因为它比存完整集合便宜太多。

4.5 Jaccard 相似度

近重复检测通常先要定义一个相似度。最常见的是 Jaccard：

$$\text{Jaccard}(A, B) = \frac{|A \cap B|}{|A \cup B|}.$$

如果两个文档的 Jaccard 相似度超过阈值，就可以视为近重复。困难在于如何在线性时间内找到这些候选对，而不是对所有文档两两比较。

4.6 MinHash

MinHash 的关键性质是：对于随机哈希函数 h ，

$$\Pr[h(A) = h(B)] = \text{Jaccard}(A, B).$$

直觉上，它像是在对集合做随机排列，然后比较哪个元素最先出现。相似集合更容易在同一个位置上“先到达”，所以碰撞概率和 Jaccard 正相关。

MinHash 的作用

它不是直接告诉你“相似不相似”，而是把相似度变成“碰撞概率”。这个概率就可以拿去做大规模近重复索引。

4.7 Locality Sensitive Hashing

LSH 的目标是把 MinHash 里的“概率相关”再放大一些，让高相似度对更容易碰撞，低相似度对更不容易碰撞。最经典的方法是把 n 个 hash 切成 b 个 band，每个 band 有 r 个 hash，满足 $n = br$ 。

如果文档相似度为 s ，那么：

$$\Pr[\text{某个 band 匹配}] = s^r, \quad \Pr[\text{至少一个 band 匹配}] = 1 - (1 - s^r)^b.$$

这就是 LSH 的阈值效应来源：当 r 变大时，曲线更陡，匹配更难；当 b 变大时，曲线向左移动，更容易匹配。

LSH 的工程含义

- 它让近重复候选对的搜索变成“先哈希分桶，再局部比较”。
- 它把原本连续的相似度问题，变成了有明显阈值的碰撞问题。
- 它适合大规模语料中的去重索引和候选召回。

5 把这一切统一起来

这一讲的核心不是某一个具体模型，而是一个非常稳定的工程模式：**先学一个粗糙但足够快的打分器，再用它筛掉不想要的数据。**

统一框架

1. 用目标数据估计一个模型、分类器或权重函数。
2. 在原始数据上对每个样本打分。
3. 按阈值、重采样或分桶规则保留样本。

如果你回看前面的例子，会发现它们几乎是同一个模板的不同实例：

- KenLM 用 $p_T(x)$ 或 perplexity 过滤；
- fastText 用 $p(T | x)$ 过滤；
- DSIR 用 $p_T(x)/p_R(x)$ 重采样；
- 语言识别、质量过滤、毒性过滤只是不同的目标定义；
- Bloom filter、MinHash、LSH 则是去重问题里的对应实现。

最重要的现实约束

这些方法的共同限制不是“数学不优雅”，而是它们都在和算力、数据规模、误杀率做权衡。工程上能跑、能扩展、能稳定复现，往往比模型是否足够精巧更重要。

6 本章小结

这一讲的主旨是：**数据处理本身就是建模问题**。只不过这里的模型不一定追求高精度，而是追求大规模、低成本、可解释和可操作。

- 过滤的本质是从原始数据中挑出更像目标数据的子集。
- KenLM、fastText 和 DSIR 是三种不同风格的实现，但都遵循“先打分，再筛选”的范式。
- 语言识别、质量过滤和毒性过滤都可以复用同一套机制。
- 去重从精确重复走到近重复后，Bloom filter、MinHash 和 LSH 就成了关键工具。

一句话总结：**模型越大不一定越好，能把数据处理做得又快又准，往往更能决定最终训练效果。**